

Rapport IA

Groupe 4 : Onitama



Membres du groupe:

- AKLI Melissa
- BALERAS Timothé
- BELASKRI Roumaissa
- BOREL Matthys
- DJEBBAR Anis
- FASLA Mohammed Ryad
- OULD BRAHIM Meriem Lyna

Tutrice de Projet : Dima Dalloul

Sommaire

Sommaire	2
Description du jeu	3
Pistes de développement et réflexions	4
Heuristique IA et Tests	11
Heuristiques et critères	11
Tests et statistiques	12
Conclusion	6

Description du jeu

L'histoire du jeu (texte issu du jeu original) :

“Sculpté dans les rochers des montagnes embrumées du vieux Japon réside l'Autel de l'Onitama. Ce lieu de purification et de formation subtile de l'esprit accueille les écoles d'arts martiaux de tout le pays. Les maîtres de ces écoles effectuent le voyage vers l'Onitama avec leurs disciples les plus prometteurs afin d'affronter leurs adversaires dans cette enceinte sacrée et prouver leur supériorité. Ces combats sont hors-normes car les esprits des animaux qui guident les combattants montrent leur voie aux maîtres et soutiennent les élèves avec une force et une dextérité venues d'un autre monde.”

Spécification

- ❖ Type de jeu: Stratégie
- ❖ Age: 14 ans et +
- ❖ Public cible: Familial / adulte
- ❖ Nombre de joueurs: 2 Durée: 15-20 minutes

Materiel

Un pion maître pour chaque joueur



Quatres pions soldats pour chaque joueur

4×

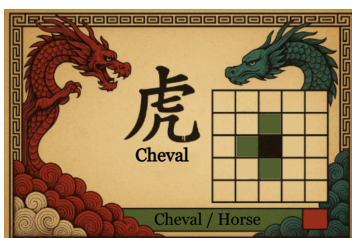


4×



Le jeu comporte 16 cartes de mouvement, dont seulement 5 sont utilisées pendant une partie

16×



Un plateau de dimensions 5×5 , avec les temples de maîtres rouge et bleu situés aux positions (0, 2) et (4, 2)



Règles

Mise en place

- ❖ Le plateau est une grille de 5×5 cases.
- ❖ Chaque joueur dispose de 1 maître (placé sur la case centrale de sa première ligne) et 4 élèves (placés sur les autres cases de la première ligne).
- ❖ Les temples sont situés aux positions (0, 2) pour le joueur rouge, et (4, 2) pour le joueur bleu.
- ❖ Il existe 16 cartes de mouvement. Au début de chaque partie :
 - Chaque joueur reçoit 2 cartes de mouvement aléatoires, affichées face visible.
 - Une 5^e carte est placée au centre du plateau. La marque colorée en bas à droite de cette carte indique quel joueur commence.

Déroulement d'un tour

1. Choix de la carte : Le joueur actif sélectionne l'une de ses deux cartes de mouvement.
2. Déplacement : Il déplace l'un de ses pions (maître ou élève) selon le motif affiché sur la carte.
 - Le carré noir au centre du motif représente la position de départ du pion.
 - Les cases colorées indiquent les destinations possibles (en relatif).
 - Le pion ne peut pas sortir du plateau, ni atterrir sur un pion allié.
 - Si le pion atterrit sur une case occupée par un pion adverse, celui-ci est capturé et retiré du jeu.
3. Échange de carte : La carte utilisée est placée à droite du plateau, face visible pour l'adversaire. Le joueur récupère la carte qui s'y trouvait précédemment et l'ajoute à sa main.

Si aucun déplacement n'est possible

Si un joueur ne peut effectuer aucun mouvement valide avec ses deux cartes, il passe son tour mais doit quand même échanger sa carte.

Objectif du jeu - Comment gagner la partie ?

Chaque joueur incarne un maître d'art martial accompagné de quatre élèves. Le but est de vaincre le maître adverse ou atteindre son temple avec votre propre maître.

Il existe deux conditions de victoire :

- **Voie de la Pierre** : capturer le maître adverse.
- **Voie du Ruisseau** : déplacer votre maître sur la case du temple adverse.

I - Pistes de développement et réflexions

Approche initiale

Avant de réfléchir à l'implémentation de l'IA, nous avons commencé par identifier la nature du jeu Onitama ainsi que ses particularités et contraintes :

- Jeu à deux joueurs
L'affrontement oppose systématiquement un joueur bleu et un joueur rouge, qui jouent à tour de rôle.
- Jeu à information complète
Chaque joueur connaît en permanence l'état exact du plateau ainsi que les cartes de déplacement détenues par son adversaire. Il n'y a ni éléments cachés ni hasard, excepté lors de la distribution initiale des cartes. Cela permet d'anticiper tous les coups possibles au prochain tour.
- Jeu à somme nulle
Les intérêts des deux joueurs sont strictement opposés : le gain de l'un est exactement l'opposé de la perte de l'autre.
- Jeu symétrique: Les deux joueurs disposent des mêmes règles, du même nombre de pièces, des mêmes possibilités de déplacement (via les cartes), et des mêmes conditions de victoire, ce qui garantit qu'aucun joueur n'est avantagé structurellement en début de partie.

Choix de la méthode d'IA

Au tout début de notre projet, nous avons envisagé d'utiliser le machine learning, voire le deep learning. Cependant, faute d'un jeu de données (dataset) de parties d'Onitama enregistrées, nous avons rapidement abandonné cette piste.

Notre choix s'est alors naturellement orienté vers l'algorithme Minimax, reconnu comme une référence pour les jeux de plateau et de stratégie à deux joueurs, à somme nulle et information complète.

L'algorithme Minimax, issu de la théorie des jeux, est utilisé pour les jeux à somme nulle entre deux joueurs à information complète. Dans ce contexte, on distingue deux joueurs : un joueur maximisant et un joueur minimisant. Cet algorithme repose sur l'exploration de l'arbre des possibilités en supposant que les deux joueurs sont rationnels, c'est-à-dire qu'ils choisissent chacun d'attaquer ou de défendre de manière optimale.

En anticipant les actions optimales de l'adversaire, on prédit son coup, puis on sélectionne le coup qui mènera à l'issue la plus favorable pour soi.

Concepts fondamentaux du Minimax

L'algorithme Minimax repose sur plusieurs concepts clés que nous avons adaptés au contexte d'Onitama :

Les nœuds

Chaque nœud représente une configuration précise du jeu d'Onitama, c'est-à-dire un instantané complet du plateau, des cartes détenues par chacun des deux joueurs, ainsi que de la carte centrale.

La notion de demi-coup

Dans Onitama, chaque action effectuée par un joueur constitue un demi-coup. Dans l'arbre des possibilités, un demi-coup correspond à une branche reliant deux nœuds, illustrant la transition d'un état du jeu à un autre suite à un seul coup joué.

La profondeur d'analyse

La profondeur d'analyse détermine combien de coups à l'avance l'IA peut anticiper. Plus cette profondeur est grande, meilleure est la qualité du jeu, mais au prix d'un temps de calcul plus long. Elle correspond à la profondeur de l'arbre de recherche et reflète la capacité de l'IA à prédire les mouvements adverses.

Le facteur de branchement

Ce paramètre correspond à l'arité d'un nœud. Dans notre implémentation, nous générons tous les successeurs possibles pour chaque nœud exploré. Onitama présente l'avantage d'un facteur de branchement modéré comparé à des jeux comme les échecs : chaque joueur ne disposant que de deux cartes à un instant donné, le nombre total de coups possibles à calculer reste gérable.

La notion d'heuristique

Dans la pratique, il est impossible d'explorer l'intégralité de l'arbre de jeu jusqu'aux positions finales à cause de la croissance exponentielle du nombre de feuille. L'algorithme Minimax repose donc sur une **fonction d'évaluation heuristique** qui estime la qualité d'une position donnée lorsque la recherche s'arrête, faute de temps ou de profondeur suffisante.

Cette fonction heuristique analyse l'état du plateau et attribue un score à chaque position selon plusieurs critères stratégiques adaptés à Onitama (sécurité du roi, contrôle du centre, mobilité, etc.).

Un algorithme de Minimax pour Onitama aura donc comme principaux paramètres la profondeur, l'heuristique, et le degré de branchement. Plus l'heuristique est fine ou plus la profondeur est grande, meilleures seront les performances.

Techniques d'optimisation implémentées

Progressive Deepening (Approfondissement itératif)

Face aux contraintes temporelles du jeu en temps réel, nous avons implémenté la technique du Progressive Deepening. Cette approche consiste à allouer un temps limité à l'IA (1 seconde dans notre cas) pendant lequel elle explore l'arbre de jeu en augmentant progressivement la profondeur d'analyse, cela correspondrait au temps de réflexion d'un joueur dans une situation réelle. L'algorithme commence par une recherche superficielle (profondeur 1), puis approfondit itérativement (profondeur 2, 3, 4...) jusqu'à épuisement du temps imparti, il est prouvé que cela n'augmente la complexité de beaucoup.

Cette technique présente un avantage crucial : elle garantit toujours qu'un coup sera disponible, sa qualité variera en fonction du temps alloué. En effet, la complexité temporelle augmente de manière exponentielle avec la profondeur, et le Progressive Deepening permet d'obtenir le meilleur compromis possible entre qualité du coup et respect des contraintes temporelles. De plus, les résultats des recherches précédentes peuvent être réutilisés pour optimiser les itérations suivantes.

Élagage Alpha-Beta

Pour optimiser davantage les performances, nous avons intégré la technique d'élagage Alpha-Beta, une amélioration significative de l'algorithme Minimax standard. Cette technique permet d'éliminer des branches entières de l'arbre de recherche qui ne peuvent pas influencer la décision finale.

Le principe repose sur le maintien de deux valeurs : Alpha (la meilleure valeur trouvée pour le joueur maximisant) et Beta (la meilleure valeur trouvée pour le joueur minimisant). Lorsque Alpha devient supérieur ou égal à Beta, cela signifie qu'une coupure peut être effectuée car les branches restantes ne pourront pas améliorer le résultat. Cette optimisation peut réduire considérablement l'espace de recherche, permettant d'explorer des profondeurs plus importantes dans le même temps imparti, sans altérer la qualité des décisions prises.

Continuation heuristique et Singular Extension

Pour contrer "l'horizon effect", phénomène où l'IA peut mal évaluer une position en raison de la limite de profondeur imposée, nous avons implémenté une continuation heuristique limitée à 5 niveaux supplémentaires. Cette technique, appelée Singular Extension, permet d'étendre la recherche au-delà de la profondeur nominale lorsqu'un coup semble significativement supérieur aux autres alternatives, c'est-à-dire calculer les successeurs d'un nœud qui semble très bon ou très mauvais.

Move Ordering (Ordonnancement des coups)

L'efficacité de l'élagage Alpha-Beta dépend fortement de l'ordre dans lequel les coups sont examinés. Nous avons donc implémenté un système de Move Ordering qui trie les coups selon leur probabilité d'être optimaux. Les coups de capture, les menaces directes sur le maître

adverse, et les mouvements vers le centre sont prioritaires dans l'exploration, maximisant ainsi les chances de coupures précoces et réduisant l'arbre de recherche.

Optimisation mémoire

Très rapidement, nous avons été confrontés à la réalité : la mémoire devient un facteur limitant majeur dès que l'on tente d'explorer de nombreux coups à grande profondeur. En effet, notre première version consistait à cloner l'objet entier pour chaque nouvelle position générée dans l'arbre Minimax, ce qui entraînait une consommation de mémoire considérable et rendait impossible l'exploration efficace de l'arbre de jeu.

Face à cette impasse, nous avons entièrement repensé notre modèle en introduisant une classe `ConfigurationCompacte` capable d'encoder chaque état du jeu de manière très compacte en mémoire. L'utilisation de tableaux de bits nous a permis de réduire drastiquement l'empreinte mémoire, passant de 100 octets pour le seul plateau à seulement 10 octets pour l'état complet du jeu.

Évolution de notre approche et premiers essais heuristiques

Au début du projet, nous avons conçu une première IA basée sur une implémentation classique du Minimax, combinée à l'élagage Alpha-Beta, au progressive deepening, et à la singular extension pour traiter certaines situations critiques.

Néanmoins, à ce stade, notre heuristique était volontairement très simple, voire naïve : elle se limitait principalement à la différence de nombre de pions entre les deux camps. Ce choix, n'avait pour but que de fournir un prototype fonctionnel afin de valider rapidement l'architecture globale de notre IA, principalement de vérifier si notre algorithme de Minimax est correctement implémenté et de repérer les points sensibles de performance et de découvrir les difficultés techniques du projet.

Très rapidement, nous avons été confrontés à la dure réalité : **la mémoire devient un facteur limitant majeur** dès que l'on tente d'explorer de nombreux coups à grande profondeur. En effet, notre première version consistait à cloner l'objet entier pour chaque nouvelle position générée dans l'arbre Minimax, ce qui entraînait une consommation de mémoire considérable et rendait impossible l'exploration efficace de l'arbre de jeu.

Face à cette impasse, nous avons entièrement repensé notre modèle. Nous avons introduit une classe `ConfigurationCompacte` capable d'encoder chaque état du jeu de manière très compacte en mémoire (utilisation de vecteurs de bits permettant de passer de 100 octets rien que pour le plateau à 10 octets)

i) Élaboration d'une heuristique multi-critères

Après avoir mis en place une version prototype basée sur une heuristique très simple permettant de valider notre implémentation du MiniMax, nous avons ouvert un débat approfondi sur la définition des critères pertinents pour l'évaluation d'une position à Onitama.

Au fil de nos discussions et itérations, nous avons identifié douze critères distincts, chacun reflétant une facette du jeu. Ce choix s'inspire à la fois des principes classiques des jeux de plateau (échecs,dames...) et des spécificités propres à Onitama.

Voici la liste de ces critères retenus :

- **Matériel**
Différence du nombre de pièces (rois et pions) entre les deux camps.
Objectif : Favoriser la capture de pièces et l'avantage matériel.
- **Mobilité**
Nombre de coups légaux disponibles pour le joueur.
Objectif : Mesurer la liberté d'action et limiter l'étouffement stratégique.
- **Centre**
Contrôle de la case centrale et de ses quatre voisines orthogonales.
Objectif : Valoriser le contrôle du centre du plateau.
- **Densité**
Répartition du nombre de pièces entre alliés et ennemis.
Objectif : Éviter la dispersion des pièces et favoriser la concentration stratégique.
- **Position des pions**
Qualité de position des pions selon leur proximité au centre.
Objectif : Encourager des pions bien placés, utiles pour l'attaque ou la défense.
- **Positionnement du roi**
Qualité de position du roi selon sa proximité au centre.
Objectif : Mesurer la sécurité et le potentiel offensif/défensif du roi.
- **Valeurs des Cartes;**
Potentiel des cartes en main, mesuré par le nombre de demi-coup possible qu'elle permettent de faire.
Objectif : Évaluer l'impact stratégique des cartes actuellement disponibles.
- **Proximité avec le Temple**
Proximité du roi allié à son temple adverse.
Objectif : Encourager le jeu offensif en direction de la victoire par arrivée au temple.
- **Alignement du Roi avec le Temple**
Présence du roi sur la colonne centrale menant au temple.
Objectif : Favoriser les lignes d'attaque directes et les menaces sur le temple adverse.

- **Nombre d'ennemis menaçant directement le roi.**
Objectif : Prendre en compte la vulnérabilité immédiate du roi.
- **Degré de protection du roi par les pions alliés situés autour de lui.**
On compte le nombre de pièces adverses autour du roi.
Objectif : Valoriser les formations défensives efficaces.
- **Proximité du roi adverse à notre propre temple**
Proximité du roi adverse à notre propre temple en calculant la distance de Manhattan entre ce dernier et le temple.
Objectif : Anticiper les menaces de victoire adverse par invasion du temple.
- **Prise du Roi Adverse, Prise du Temple Adverse**
Cela donne naturellement un score de +infini car cela correspond à une victoire.
- **Prise du Roi allié, Prise du Temple allié**
Analogue à ce dernier avec un score de -infini.

ii) Réduction expérimentale des critères

Cependant, au fil de nos expérimentations, nous avons constaté que certains critères étaient soit redondants et très fortement corrélés, soit peu discriminants dans la pratique. Nous avons donc progressivement affiné et simplifié notre fonction d'évaluation, en ne conservant que les critères réellement déterminants pour la performance de l'IA.

Enfin, chaque critère conservé a été pondéré à l'aide d'un coefficient ajustable, reflétant son importance stratégique dans l'évaluation globale d'une position.

La fonction d'évaluation finale consiste donc en une combinaison linéaire pondérée de ces critères, ce qui permet d'équilibrer finement leur influence et d'ajuster le comportement de l'IA selon l'expérience ou des méthodes d'optimisation automatique.

iii) Protocole expérimental

Pour sélectionner et calibrer au mieux nos critères, nous avons adopté un protocole expérimental simple mais efficace :

Nous avons développé un programme de benchmark textuel permettant de faire s'affronter deux IA dans des milliers de parties simulées automatiquement. Ce code, optimisé pour le multithreading, permet de lancer plusieurs parties en parallèle afin d'obtenir rapidement des statistiques sur les performances relatives de chaque version d'heuristique.

La démarche était alors la suivante :

- Nous choisissons un ensemble de critères à tester (parmi la liste précédente).
- Nous modifions manuellement la pondération (poids des critères) d'une nouvelle version de l'heuristique.

- Nous confrontons cette nouvelle version de l'IA contre la version précédente, sur un grand nombre de parties (IA vs IA).
- Si la nouvelle version montrait une amélioration significative du taux de victoire, elle était adoptée, sinon nous testions une autre combinaison de poids ou de critères.
- Ce processus d'ajustement manuel des pondérations s'est avéré très instructif pour comprendre l'importance relative de chaque critère.

Nous avons également tenté d'appliquer des méthodes d'optimisation automatique des poids, comme l'algorithme SPSA (Simultaneous Perturbation Stochastic Approximation), mais le choix des hyperparamètres s'est révélé difficile et les contraintes de temps d'exécution (dus à la lourdeur des simulations) limitaient la pertinence de la méthode.

iv) Variation du niveau de difficulté

Nous nous sommes en réalité intéressés au développement de trois niveaux d'IA, en nous inspirant de ce qui se fait généralement dans ce type de jeu classique :

- IA facile : Elle joue aléatoirement parmi les coups qui ne permettent pas à l'adversaire de gagner en un coup ou de capturer un pion. Sinon, si une victoire ou une capture est possible, elle la saisit.
- IA intermédiaire : Conçue à partir d'une heuristique et d'une implémentation simplifiée du Minimax, cette IA avait initialement vocation à être l'IA difficile. Cependant, après avoir développé une IA plus performante, nous avons rétrogradé celle-ci au rang d'intermédiaire.
- IA difficile : Elle repose sur une heuristique différente et une implémentation du Minimax intégrant toutes les optimisations que nous avons mises en place.

v) Perspectives d'amélioration

Pour optimiser davantage notre système, plusieurs pistes d'amélioration pourraient être explorées :

Élagage heuristique : L'implémentation d'un élagage plus agressif basé sur des heuristiques d'Onitama pourrait permettre d'éliminer prématurément des branches manifestement inférieures, libérant ainsi du temps de calcul pour les variantes prometteuses.

Fichiers d'ouvertures et de finales : En s'inspirant des concepts utilisés aux échecs, on pourrait développer des bibliothèques de positions d'ouverture et de finale. Ces bases de données permettraient à l'IA de jouer instantanément les premiers coups selon des principes préétablis, et de résoudre parfaitement les positions de fin de partie avec un nombre réduit de pièces. Cela permettrait de réduire considérablement le temps de calcul lors des phases critiques du milieu de jeu. En implémentant ce système, il serait possible d'explorer les coups à très grande profondeur, puisque le temps ne sera plus un facteur limitant.

Utilisation du Multithreading : Il serait également possible de déléguer le calcul de chacune des branches à des threads différents pour peut être pleinement les capacités de notre processeur.

Utilisation de Monte-Carlo: Tester la pertinence d'une configuration en faisant plusieurs simulation aléatoire.

II - Heuristiques, IA et Tests:

Heuristique et critères

i) Heuristique Difficile:

L'heuristique difficile attribue des scores extrêmes en cas de victoire ou de défaite, que ce soit par capture du roi ou par arrivée au temple. Ces scores sont identiques en valeur absolue, mais de signe opposé, afin de ne favoriser aucune condition de victoire par rapport à l'autre.

En dehors de ces cas terminaux, l'heuristique prend en compte plusieurs critères stratégiques :

- la **sécurité du roi**, en évaluant s'il peut être menacé à court terme,
- le **contrôle du centre du plateau**,
- la **distance entre le roi et le temple adverse**,
- et la **puissance de la formation**, mesurée par la protection du roi allié et le regroupement des pièces adverses et alliés.

ii) Heuristique Intermédiaire:

L'heuristique intermédiaire traite les états terminaux avec une approche asymétrique, attribuant des valeurs différentes selon le type de victoire ou défaite. Les victoires (capture du roi ou arrivée au temple) reçoivent un bonus modeste, tandis que les défaites sont fortement pénalisées, créant une priorité défensive marquée. Les menaces immédiates sur le roi sont sanctionnées par un malus critique, établissant la sécurité comme priorité absolue.

Au-delà des situations critiques, l'heuristique évalue la position selon **quatre critères équipondérés** :

- la **progression vers l'objectif**, mesurée par la distance du roi au temple adverse,
- l'**avantage matériel**, calculé par le nombre de pions capturés,
- l'**équilibre des forces**, représenté par la différence de pièces entre les deux camps,
- et la **sécurité immédiate**, déterminée par l'absence de menaces directes sur le roi.

Cette approche privilégie une évaluation où chaque facteur contribue uniformément au score final, favorisant un style de jeu mettant la défense au premier-plan tout en ne négligeant pas l'attaque.

Tests et Statistiques

Afin de valider la qualité et la cohérence de nos IA, nous avons mené deux types de tests complémentaires. D'une part, des tests automatisés opposant nos IA entre elles (IA vs IA), permettant d'obtenir un grand volume de résultats objectifs et reproductibles. D'autre part, des tests contre de vrais joueurs, afin de confronter nos IA à des comportements humains et évaluer leur crédibilité dans des conditions réelles. Ces deux approches étaient nécessaires pour s'assurer que les niveaux de difficulté que nous avons conçus se traduisent de manière fiable en pratique.

i) Test Automatisés

Nous avons confronté nos trois IA deux à deux, dans le but de valider nos résultats et de vérifier si les scores obtenus en IA vs IA reflètent une hiérarchie cohérente. L'objectif était que l'IA difficile se comporte comme un joueur très fort, l'IA intermédiaire comme un joueur moyen, et l'IA facile comme un joueur débutant. Les résultats vont dans ce sens, ce qui confirme la pertinence de notre approche.

Nous observons ainsi que l'IA difficile bat systématiquement l'IA facile, avec seulement 3 % de défaites, probablement dues à des facteurs aléatoires. Cela s'explique par le fait que l'IA facile, bien que simple, n'est pas totalement aléatoire et peut parfois profiter d'une opportunité favorable.

Concernant le duel entre l'IA difficile et l'IA intermédiaire, les résultats sont similaires, bien que moins extrêmes : l'IA difficile gagne environ 85 % du temps. Cela s'explique par le fait que l'IA intermédiaire reste stratégique et possède une certaine capacité de réflexion.

Enfin, l'IA intermédiaire bat l'IA facile dans 91 % des cas, ce qui s'aligne parfaitement avec nos attentes et confirme la cohérence de notre classement.

À noter que le jeu Onitama est symétrique, et que le fait de jouer rouge ou bleu n'influence pas les performances. De plus, nous avons alloué à chaque IA une seconde de réflexion par coup, ce qui permet d'atteindre une profondeur de 4 à 5 pour l'IA difficile, et de 5 à 8 pour l'IA facile (en raison d'un arbre de recherche plus simple). Le tableau ci-dessous résume bien nos résultats.

Joueur 1	Joueur 2	Nombre de partie	Nombre de victoires (Joueur 1)	Nombre de victoires (Joueur 2)	% de victoires (Joueur 1)	% de victoires (Joueur 2)
<i>Difficile</i>	<i>Facile</i>	<i>1000</i>	<i>971</i>	<i>29</i>	<i>97,1</i>	<i>2,9</i>
<i>Difficile</i>	<i>Moyenne</i>	<i>1000</i>	<i>854</i>	<i>146</i>	<i>85.4</i>	<i>14.6</i>
<i>Moyenne</i>	<i>Facile</i>	<i>1000</i>	<i>912</i>	<i>88</i>	<i>91.2</i>	<i>8.8</i>

Tableau récapitulatif des résultats des parties IA vs IA

ii) Validation par de vrai joueurs:

Afin de compléter notre évaluation automatisée, nous avons également soumis nos trois IA à des parties contre de vrais joueurs humains, de niveaux variés. Cette phase de test avait pour objectif de vérifier si le comportement observé en IA vs IA se confirmait face à des stratégies humaines, parfois imprévisibles. Les retours recueillis confirment globalement la hiérarchie que nous avons établie : l'IA facile est accessible et battable même par des débutants, l'IA intermédiaire représente un bon défi pour des joueurs occasionnels, tandis que l'IA difficile s'est montrée redoutable, remportant la majorité de ses parties, même contre des joueurs expérimentés. Cette validation par des utilisateurs réels renforce la crédibilité de notre échelle de difficulté, et montre que nos IA offrent une expérience de jeu progressive et équilibrée.